

localhost:8888/notebooks/machine_learning_regression

Jupyter Machine_learning_Regression Last Checkpoint: 20 minutes ago

File Edit View Run Kernel Settings Help

Code Python 3

```
[ ]: import pandas pd
      from matplotlib import pyplot as plt
      from sklearn.model_selection import train_test_split
      from sklearn.linear_model import LinearRegression
      from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

[ ]: python 100
      c# 0
      Average performance = 50
```

http://localhost:8890/notebooks/Machine_learning_Regression.ipynb

Jupyter Machine_learning_Regression Last Checkpoint: 42 minutes ago

Edit View Run Kernel Settings Help

Code Python 3 (ipykernel)

```
df.describe() # Summary stats
```

[5]:

	Experience_Years	Trainings_Completed	Age	Salary
count	200.000000	200.000000	200.000000	200.000000
mean	7.720000	4.725000	40.430000	28857.674250
std	4.431681	2.663616	11.243987	11531.501182
min	0.000000	1.000000	20.000000	-1371.690000
25%	4.000000	2.000000	31.000000	20930.947500
50%	8.000000	4.500000	41.500000	29145.305000

→ ↻ ↗ http://localhost:8890/notebooks/Machine_learning_Regression.ipynb 🔍 ☆ ... 🔍 Search

 **jupyter** Machine_learning_Regression Last Checkpoint: 1 hour ago 

File Edit View Run Kernel Settings Help Trusted

⊞ ✂ 📄 📄 ▶ ■ ↺ ▶▶ Code ▾ JupyterLab 🔗 🛠 Python 3 (ipykernel) ○ ☰ ☰

```
[4]: # Load the dataset
df = pd.read_csv(r'C:\Users\DELL LATITUDE E7470\Downloads\employee_salary_dataset.csv')
# show first five rows
df.head()
```

[4]:

	Experience_Years	Trainings_Completed	Age	Salary
0	6	7	44	26386.89
1	3	1	43	13696.15
2	12	4	32	33655.75
3	14	4	26	38177.61
4	10	5	55	34344.67

Salaries

```
# selecting dependant(y) and independant variable(x)
X = df[['Experience_Years', 'Trainings_Completed', 'Age']]
y = df["Salary"]
```

```
# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=42)
```

```
# Train the regression model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
# check model coefficiene for interpretation
model.coef_, model.intercept_
```

```
(array([2518.09324749, 1169.03006933, 105.38625319]), -304.3133137586956)
```

```
# make a real prediction using the model
sample = [[5, 3, 30]]
model.predict(sample)
```